

Bahria University,

Karachi Campus



**COURSE: SEL-448 SOFTWARE APPLICATIONS FOR MOBILE
DEVICES**
TERM: SPRING 2026, CLASS: BSE- 6 (A/B/C)



Group Members	Enrollment No.
Maria Khan (Lead)	02-131232-114
Muhammad Umer Qureshi	02-131232-108
Muhammad Umer	02-131232-090

Engr. Adnan-ur-Rehman/Engr. Saniya Sarim

Signed

Remarks:

Score:

1.	Table of Contents	2
2.	System Architecture and Design	3
2.1	Architectural Overview	3
2.2	Layered Architecture	3
2.3	Directory Structure	5
2.4	Navigation Architecture	6
2.5	State Management	6
2.6	Service Layer	7
2.7	Data Models	8
2.8	Persistence Layer	9
2.9	External API Integration	9
2.10	Theming System	10
3.	Implementation Details (Code Explanation)	12
3.1	Application Entry Point (main.dart)	12
3.2	Prayer Times — Home Screen	13
3.3	Qibla Compass Screen	14
3.4	Tasbeeh Counter Screen	15
3.5	Azan Reminders Screen	16
3.6	Streak Tracker Screen	17
3.7	Prayer Guidance Screen	18
3.8	Mosque Map Screen	19
3.9	Notification Service	20
3.10	Splash Screen	21
3.11	Settings Screen	21
4.	Output Screenshots	22
5.	Challenges and Solutions	24

2. System Architecture and Design

2.1 Architectural Overview

Awwab (Smart Namaz Companion) is a cross-platform Islamic mobile application built using the Flutter framework with the Dart programming language. The application follows a layered, service-oriented architecture that cleanly separates concerns into distinct tiers: the UI Layer, State Management Layer, Service Layer, and Data/Persistence Layer. This design promotes maintainability, testability, and a clear data flow throughout the application.

The application is designed to serve as an all-in-one digital companion for Muslims who wish to maintain their daily prayer routine. It integrates real-time prayer time calculations, step-by-step Salah guidance, a Qibla compass, a digital Tasbeeh counter, gamified streak tracking, Azan notification scheduling, a nearby mosque finder, and daily Hadith — all within a single, cohesive, and beautifully themed interface. The name "Awwab" (أَوَّاب) is Arabic for "one who frequently returns to Allah," reflecting the app's mission to encourage consistent prayer habits.

The application targets six platforms from a single Dart codebase: Android (primary), iOS, Web (PWA), macOS, Linux, and Windows. The Flutter framework enables this true cross-platform reach, allowing the team to maintain a unified codebase while delivering native-quality experiences on each platform.

2.2 Layered Architecture

The application is organized into four primary layers, each with a well-defined responsibility:

Layer 1 — UI Layer (Screens)

The topmost layer consists of Flutter screen widgets that render the user interface. Each screen is implemented as a StatefulWidget and is responsible for presenting data to the user and capturing user interactions. The app features eight main screens accessible through a bottom navigation bar and a side drawer: Home Screen (Prayer Times), Qibla Compass, Tasbeeh Counter, Reminders, Streak Tracker, Prayer Guidance, Mosque Map, and Settings. Multiple custom CustomPainter classes are used throughout the app for rendering complex visuals such as the compass rose, tasbeeh beads, progress rings, and decorative wave patterns.

Layer 2 — State Management Layer (Provider)

The application uses the Provider package for global state management. Two providers are registered at the root: a LanguageProvider (extending ChangeNotifier) that manages the app-wide language state, enabling instant switching between English and Urdu without restarting the app; and a ThemeNotifier (also extending ChangeNotifier) that manages the light/dark theme mode, persisting the user's preference to SharedPreferences. Both are injected via MultiProvider in main.dart and consumed throughout the widget tree using Consumer2. Local screen-level state (such as counter values, selected prayers, and loading indicators) is managed using standard Flutter StatefulWidget with setState.

Layer 3 — Service Layer

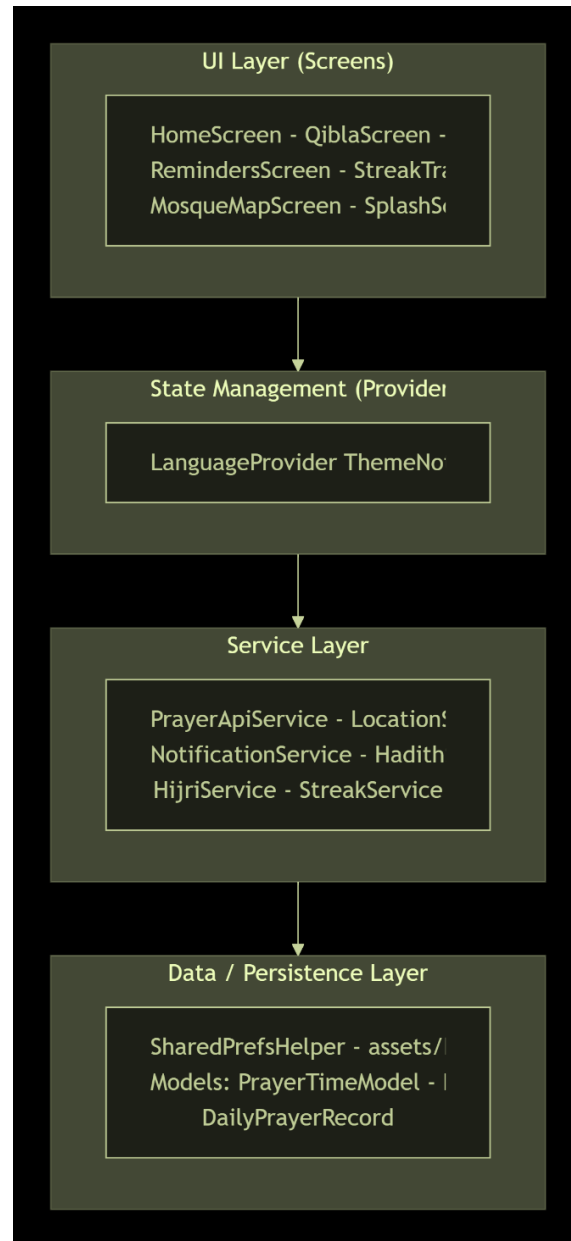
The service layer contains stateless utility classes that encapsulate business logic, API communication, and device interactions. Each service has a single responsibility: PrayerApiService handles HTTP communication with the Aladhan REST API; LocationService wraps the Geolocator plugin for GPS position retrieval; NotificationService manages local notification scheduling with a dual-channel system (system sound for prayer banners and custom Azan sound for manual reminders); HadithService loads and parses the bundled Hadiths JSON asset; HijriService computes and formats today's Islamic calendar date; and StreakService manages the reading and writing of daily prayer completion records with an in-memory cache for performance.

Layer 4 — Data/Persistence Layer

The data layer handles persistent storage and data modeling. All user preferences, notification toggles, theme/language settings, and streak records are stored locally using the SharedPreferences plugin (key-value storage). Data models —

PrayerTimeModel, HadithModel, and DailyPrayerRecord — provide structured representations of the app's domain objects with toJson() and fromJson() serialization methods. The Hadith dataset is stored as a bundled JSON asset file (assets/hadiths.json) loaded once at runtime. Prayer times are also cached to SharedPreferences for instant display on app launch before fresh data is fetched from the API. No external database or cloud service is required, ensuring full offline capability for stored data.

Architecture Diagram:



2.3 Directory Structure

The project follows a well-organized directory structure under the lib/ folder, separating code by responsibility:

```
lib/  
|-- main.dart  
|-- models/  
|   |-- daily_prayer_record.dart  
|   |-- hadith_model.dart  
|   +-- prayer_time_model.dart  
|-- providers/
```

```

|   +-- language_provider.dart
|-- screens/
|   |-- home_screen.dart
|   |-- main_wrapper.dart
|   |-- prayer_guidance_screen.dart
|   |-- guidance_screen.dart
|   |-- qibla_screen.dart
|   |-- reminders_screen.dart
|   |-- settings_screen.dart
|   |-- streak_tracker_screen.dart
|   |-- tasbeeh_screen.dart
|   |-- mosque_map_screen.dart
|   +-- splash_screen.dart
|-- services/
|   |-- hadith_service.dart
|   |-- hijri_service.dart
|   |-- location_service.dart
|   |-- notification_service.dart
|   |-- prayer_api_service.dart
|   +-- streak_service.dart
+-- utils/
    |-- shared_prefs_helper.dart
    +-- theme.dart

```

2.4 Navigation Architecture

Awwab employs two complementary navigation patterns:

Bottom Navigation Bar (Persistent Shell):

The MainWrapper widget serves as the persistent navigation shell, hosting a BottomNavigationBar with four primary tabs: Home (Namaz/Prayer Times) at index 0, Qibla Compass at index 1, Tasbeeh Counter at index 2, and Reminders at index 3. An IndexedStack is used to keep all visited screens alive in the widget tree, preventing re-initialization when switching tabs. Tab labels are bilingual, switching between English and Urdu via the LanguageProvider.

Push Navigation (Secondary Screens):

Secondary screens are navigated to using MaterialPageRoute and Navigator.push, including: the Settings screen (accessible from the side drawer and Reminders screen), Prayer Guidance screen (pushed when tapping a prayer time card on the Home screen), Streak Tracker (accessible from the Home screen), and the Mosque Map screen. The SplashScreen serves as the initial route, transitioning to the MainWrapper using a custom PageRouteBuilder with FadeTransition for a smooth cross-fade animation.

2.5 State Management

The application employs the Provider package for managing app-wide state. Two global providers are registered at the root of the widget tree via MultiProvider:

- LanguageProvider:

A ChangeNotifier that holds a language code (defaulting to English). When the user selects a language from the dropdown in Settings, the provider calls setLanguage() followed by notifyListeners(), which triggers an immediate rebuild of all widgets that depend on the language state. This enables instant, app-wide language switching between English and Urdu without restarting the application. Every user-facing string in the app uses a ternary expression checking the isUrdu flag.

ThemeNotifier: A ChangeNotifier that manages the light/dark theme preference. It reads the initial theme mode from SharedPreferences on startup and persists changes when the user toggles the theme. The toggleTheme() and setSystemTheme() methods update the ThemeMode and notify listeners, causing the entire MaterialApp to rebuild with the new theme. Both providers are consumed by the root widget using Consumer2 for efficient rebuilds.

Local screen-level state — such as the Tasbih counter value, selected prayer for guidance, loading indicators, countdown timers, and form inputs — is managed using standard Flutter StatefulWidget with setState. This keeps the state management simple and appropriate for the current scale of the application.

2.6 Service Layer

Each service is a stateless class with static or instance methods, encapsulating a single area of concern:

Service	Responsibility
PrayerApiService	Performs HTTP GET requests to the Aladhan API (api.aladhan.com/v1/timings) with GPS coordinates and ISNA calculation method; parses the JSON response into a PrayerTimeModel with a 10-second timeout and graceful error handling.
LocationService	Wraps the Geolocator plugin to fetch the device's current GPS latitude and longitude; handles permission denial gracefully with fallback error messages and retry options.
NotificationService	Static utility class managing two notification channels: "azan_channel" for prayer time banners with system sound, and "reminder_azan_channel" for manual reminders with custom Azan audio. Uses android_alarm_manager_plus with a top-level @pragma callback for background Azan playback even when the app is closed.
HadithService	Loads and parses the bundled assets/hadiths.json file; returns a daily-rotated HadithModel based on the day-of-year modulus operation for daily variety.
HijriService	Uses the hijri_date package to compute today's Hijri (Islamic) calendar date and format it in both English and Urdu for display in the side drawer.
StreakService	Manages prayer streak data with an in-memory cache for performance. Reads from SharedPreferences on first access, then serves from RAM. Supports legacy format migration, fire-and-forget writes, and streak calculations (current/best).

2.7 Data Models

The application defines three core data models:

PrayerTimeModel:

Holds the five daily prayer times (Fajr, Dhuhr, Asr, Maghrib, Isha) as String values parsed from the Aladhan API response. Each time is stored in 24-hour format (HH:mm) for consistent parsing throughout the application. The model includes a fromJson() factory constructor that extracts prayer times from the API's response structure.

HadithModel:

Represents a single Hadith with fields for: arabic (original Arabic text), englishText (transliteration), urduText (Urdu script), englishTranslation, urduTranslation, englishTafseer (commentary), urduTafseer, and reference (book and hadith number). This model supports the bilingual display of daily Hadiths in the side drawer.

DailyPrayerRecord:

Tracks per-day prayer completion with: date (String in "YYYY-MM-DD" format) and five boolean fields (fajr, dhuhr, asr, maghrib, isha) indicating whether each prayer was performed. The model includes toJson() and fromJson() methods for serialization. All dates are normalized to midnight for consistent map keys in the StreakService cache.

2.8 Persistence Layer

All persistent data is stored locally on the device using the SharedPreferences plugin (key-value storage). No external database, cloud storage, or user account is required. This design ensures full offline functionality and respects user privacy. The persistence strategy uses a two-tier approach: an in-memory cache for frequently accessed data like streak records, and SharedPreferences as the durable backing store.

Key	Type	Purpose
themeMode	String	Stores the user's theme preference (light/dark/system)
language	String	Stores the user's language preference (en/ur)
notify_fajr ... notify_isha	bool	Individual notification toggles per prayer
prayer_records	JSON Map	All prayer completion records keyed by date
current_streak	int	Cached current streak count
best_streak	int	Cached best streak count
cached_prayer_times	JSON String	Last fetched prayer times for instant display
manual_reminders	JSON String	User-created manual reminder entries

The Hadith dataset is stored as a bundled JSON asset (assets/hadiths.json) and is loaded once at runtime. The SharedPrefsHelper utility class provides a clean abstraction over SharedPreferences, including methods for caching and retrieving prayer times.

2.9 External API Integration

Aladhan Prayer Times API:

The application integrates with the Aladhan API (<https://api.aladhan.com>) for real-time prayer time calculation. This is a free, open API that requires no API key, making it ideal for a community-focused application.

Parameter	Description
Base URL	http://api.aladhan.com

Endpoint	GET /v1/timings/{date}
latitude	Device GPS latitude (decimal degrees)
longitude	Device GPS longitude (decimal degrees)
method	2 (Islamic Society of North America calculation method)
Response	JSON with data.timings containing prayer times in 24h format
Timeout	10 seconds

Overpass API (OpenStreetMap):

The Mosque Map feature uses the Overpass API to query nearby mosques from OpenStreetMap data. The query searches for nodes tagged as "amenity=place_of_worship" with "religion=muslim", as well as "amenity=mosque" and "building=mosque" within a dynamically expanding radius. The search starts at 3km and progressively expands to 10km, 25km, and 50km until at least 3 mosques are found.

Nominatim Geocoding API:

Used for location search (forward geocoding) and reverse geocoding to obtain human-readable addresses. Location search includes a 600ms debounce timer to avoid excessive API calls while the user is typing.

OSRM Routing API:

The Open Source Routing Machine (OSRM) API at router.project-osrm.org provides walking and driving route calculations between the user's location and selected mosques. Routes are rendered as polylines on the map with distance and estimated travel time.

2.10 Theming System

The application features a comprehensive theming system defined in `theme.dart` through a centralized `AppColors` class and `AppTheme` utility. The color palette uses professional slate-and-olive tones with rounded corners (12-16px) and subtle elevations:

Color Constant	Hex Code	Usage
primaryMuted	#748469	Primary olive green for headers, buttons, accents
secondaryMuted	#ABB290	Lighter green for secondary elements
accentMuted	#AAC1B1	Soft accent for highlights and decorations
warmLight	#F9EAD7	Warm cream for light mode backgrounds
darkBackground	#121212	True dark for dark mode backgrounds
darkSurface	#1E1E1E	Dark surface for cards in dark mode
darkCard	#2C2C2C	Slightly lighter dark for elevated elements
darkPrimaryMuted	#8A9B7F	Adapted olive green for dark mode primary

successGreen	#4CAF50	Checkmarks and success states
errorRed	#E53935	Error states and alerts

Typography uses Google Fonts with Plus Jakarta Sans for English text and Noto Sans Arabic for Arabic/Urdu text. A `_buildTextTheme()` method applies the font across all 12 Material text styles. Both light and dark ThemeData configurations are built using these constants, with customized AppBarTheme, CardTheme, ElevatedButtonTheme, InputDecorationTheme, and BottomNavigationBarTheme for visual consistency.

2.10.1 Android Permissions

Permission	Purpose
RECEIVE_BOOT_COMPLETED	Reschedule alarms after device reboot
WAKE_LOCK	Keep CPU awake for alarm delivery
SCHEDULE_EXACT_ALARM	Precise prayer time notifications
ACCESS_FINE_LOCATION	GPS for prayer times and Qibla direction
ACCESS_COARSE_LOCATION	Fallback location when GPS is unavailable
ACCESS_BACKGROUND_LOCATION	Location access when app is in the background
INTERNET	API calls to Aladhan, Overpass, Nominatim, and OSRM
ACCESS_NETWORK_STATE	Network connectivity check
POST_NOTIFICATIONS	Android 13+ notification permission
VIBRATE	Haptic feedback for Tasbih counter and notifications

3. Implementation Details (Code Explanation)

This section provides a detailed explanation of the code implementation for each major component of the Awwab application, covering the key classes, algorithms, design patterns, and external dependencies used throughout the codebase.

3.1 Application Entry Point (main.dart)

The application's entry point is the `main()` function in `main.dart`, which performs the following initialization sequence:

- Calls `WidgetsFlutterBinding.ensureInitialized()` and `FlutterNativeSplash.preserve()` to hold the native splash screen during async setup.
- Initializes the `NotificationService`, which also initializes `AndroidAlarmManager` for background alarm scheduling, and creates the notification channels.
- Reads the saved theme mode from `SharedPreferences` to restore the user's last theme preference.
- Wraps the widget tree with `MultiProvider`, exposing both `ThemeNotifier` and `LanguageProvider` to all descendant widgets.
- Uses `Consumer2` to listen to both providers simultaneously, rebuilding the `MaterialApp` with updated theme and language settings.
- Configures `MaterialApp` with `AppTheme.lightTheme` and `AppTheme.darkTheme` from the centralized theme utility.
- Sets the `SplashScreen` as the initial route, which handles the animated transition to the main application.

The `MyApp` root widget is implemented as a `StatelessWidget` since theme and language state are managed externally by providers. The `Consumer2` pattern ensures efficient rebuilds — only the portions of the widget tree that depend on theme or language state are rebuilt when either changes.

3.2 Prayer Times — Home Screen (home_screen.dart)

The `HomeScreen` is the central hub of the application, implemented as a `StatefulWidget` spanning approximately 780 lines of code. It serves as the primary landing screen after the splash animation.

Two-Phase Data Loading Strategy:

The screen employs a two-phase loading approach: (1) on initialization, it immediately loads cached prayer times from `SharedPreferences` via `PrefsHelper.getCachedPrayerTimes()` for instant display, and (2) after a 500ms delay, it fetches fresh prayer times from the Aladhan API using the device's GPS coordinates. This ensures the user sees prayer times instantly, even before the network request completes.

Next Prayer Algorithm:

The next prayer determination iterates through the five prayers chronologically. For each prayer, the time string is parsed into a `DateTime` object using `DateTime.parse()`. The first prayer whose time is after `DateTime.now()` is identified as the "next" prayer and highlighted with a distinct visual treatment — a `primaryColor` border, elevated shadow, and an "Up next" label. If all five prayers have passed for the day, the method wraps around to tomorrow's Fajr. A custom `_formatTo12Hour()` function converts 24-hour time strings to 12-hour AM/PM format for user-friendly display.

Countdown Timer:

A `Timer.periodic` running at 1-second intervals continuously calculates the remaining `Duration` to the next prayer and formats it as `HH:mm:ss`. The timer auto-recalculates when the countdown reaches zero, advancing to the next prayer. The countdown is displayed prominently in a hero card with a gradient background and a custom `_HomeMasjidBackdropPainter` that draws a decorative wave-path mosque silhouette using `quadraticBezierTo` curves.

Side Drawer:

The home screen features a slide-out drawer containing: an app branding header; an Islamic Calendar card displaying today's Hijri and Gregorian dates computed by HijriService; a daily Hadith card loaded via HadithService showing Arabic text, English/Urdu translation, Tafseer (commentary), and reference — with the Hadith rotating daily based on a day-of-year modulus operation; and a quick link to Settings.

Notification Integration:

After fetching fresh prayer times, the home screen calls `NotificationService.scheduleAzanAlarms()` to schedule background Azan notifications for all five prayers, and `scheduleDailyHadithNotification()` to schedule a daily Hadith notification at 8:00 AM.

3.3 Qibla Compass Screen (qibla_screen.dart)

The QiblaScreen provides a real-time Qibla direction finder using the device's compass sensor and GPS location, implemented in approximately 750+ lines with custom painting.

Qibla Bearing Calculation:

The Qibla direction is calculated using a Haversine-based bearing formula. The app computes the exact angle from the user's current GPS position to the Kaabah coordinates (21.4225 degrees N, 39.8262 degrees E) using the formula: $\text{bearing} = \text{atan2}(\sin(\text{deltaLong}) \times \cos(\text{kaabaLat}), \cos(\text{userLat}) \times \sin(\text{kaabaLat}) - \sin(\text{userLat}) \times \cos(\text{kaabaLat}) \times \cos(\text{deltaLong}))$. This bearing is combined with the real-time compass heading from the flutter_compass plugin to orient the compass needle toward Makkah.

Custom Compass Rendering:

The QiblaCompassPainter (CustomPainter) renders a detailed compass rose with cardinal directions (N, S, E, W), degree markings at regular intervals, an animated Qibla direction needle, a Kaaba visualization at the Qibla bearing, and alignment indicators. The rotation is animated smoothly using `AnimatedRotation` and `TweenAnimationBuilder` for interpolated transitions. The screen includes a calibration detection system that monitors compass accuracy and displays a "figure-8" calibration dialog when low accuracy is detected.

3.4 Tasbeeh Counter Screen (tasbeeh_screen.dart)

The TasbeehScreen is one of the largest screens at approximately 800 lines, implementing a digital prayer bead counter with two tabs managed by a `TabController`:

Tab 1 — Dhikr Counter:

Features a large circular tap button that increments the count on each press. A custom `_RingPainter` (CustomPainter) draws a circular progress ring with `StrokeCap.round` that fills as the count approaches the target value. Haptic feedback is triggered on each tap via `HapticFeedback.mediumImpact()`, and a pulse animation (`ScaleTransition`) plays on the button. Five preset Dhikr options are presented in a `_DhikrPickerSheet` modal bottom sheet: SubhanAllah (target 33), Alhamdulillah (target 33), AllahuAkbar (target 34), Astaghfirullah (target 100), and La ilaha illallah (target 100). Users can also set a custom target through a numeric input dialog. A congratulatory completion dialog appears when the target is reached.

Tab 2 — 99 Names of Allah (Asma ul Husna):

Displays a hardcoded list of all 99 Names of Allah with Arabic text, English transliteration, English meaning, and Urdu meaning. A live search feature filters the list in real time using `toLowerCase().contains()` matching across all language fields. Each name is displayed as a card with large Arabic text in a serif font. The screen uses glass-morphism effects on cards with `BackdropFilter` and `ClipRRect`.

3.5 Azan Reminders Screen (reminders_screen.dart)

The RemindersScreen (approximately 600 lines) manages Azan notification scheduling with a flexible, user-friendly interface.

Manual Reminder System:

Unlike a simple toggle-per-prayer approach, the Reminders screen implements a full manual reminder system. Reminders are stored as JSON in SharedPreferences under the key "manual_reminders". Adding a reminder follows a two-step UX: (1) a time picker dialog for time selection, and (2) a modal bottom sheet (`_LabelPickerSheet`) for label selection from 9 predefined options (5 prayers + Tahajjud, Du'a, Quran, Custom) with icons and bilingual English/Urdu labels. When "Custom" is selected, a TextField appears for free-text label input.

Scheduling and Management:

Reminders are scheduled via `NotificationService.scheduleManualReminder()` with auto-advance logic (if the selected time has passed today, the reminder is scheduled for tomorrow). Each reminder receives a unique ID generated from `DateTime.now().millisecondsSinceEpoch`. Reminders can be deleted via a confirmation dialog or by swiping with the Dismissible widget (endToStart direction). A summary strip shows "X Upcoming" and "Y Passed" chip counts. The FloatingActionButton uses a ScaleTransition with easeOutBack curve.

3.6 Streak Tracker Screen (`streak_tracker_screen.dart`)

The StreakTrackerScreen (approximately 570 lines) implements a gamified habit tracking system with a GitHub-style contribution heatmap.

Calendar Heatmap:

The screen uses the `table_calendar` package with custom CalendarBuilders. A `_heatmapColor()` function maps prayer completion count (0-5) to opacity levels (0.05 to 0.78) of the primary color, creating a visual heatmap similar to GitHub's contribution graph. Days with higher prayer completion appear more saturated.

Streak Algorithm:

Streak calculations are delegated to StreakService. The current streak algorithm counts backward from today — a day counts toward the streak only if ALL five prayers are marked as completed. The moment a day with fewer than five completed prayers is encountered, the streak breaks. The best streak algorithm performs a forward scan through all sorted dates, tracking the longest consecutive run of fully completed days. Both values are cached in SharedPreferences to avoid recalculation on every load.

Daily Prayer Card:

Selecting a day on the calendar reveals a daily card showing the date, a LinearProgressIndicator with color coding (green for all 5, orange for 3-4, primary for 0-2), a completion badge (X/5), and five toggle rows for each prayer. Toggling a prayer uses `HapticFeedback.selectionClick()` and immediately persists the change. Entrance animations include FadeTransition on the header (600ms) and SlideTransition on cards (500ms).

3.7 Prayer Guidance Screen

Guidance Screen (`guidance_screen.dart`):

The GuidanceScreen serves as a comprehensive Islamic dua and dhikr hub with 10 sections covering: Basic Tasbeeh, Morning & Evening Adhkar, Comprehensive Dhikr, Before & After Salah, Special Situations, Difficult Situations, Firmness of Heart, Depression/Anxiety, Protection, Quranic Duas, and Istikhara. All content is hardcoded with Arabic text, English/Urdu translations, and Hadith references. Features include real-time search across all language fields, expandable accordion-style sections, and a copy-to-clipboard feature for Arabic text.

Prayer Guidance Screen (`prayer_guidance_screen.dart`):

The PrayerGuidanceScreen (StatelessWidget) receives a prayerName parameter and displays three main sections: (1) a Rakat Breakdown showing counts for Sunnah Muakkadah, Fardh, Sunnah Ghair Muakkadah, and Nafl; (2) Step-by-Step Instructions with numbered markers; and (3) optional Notes on timing and common considerations. All five prayers are fully covered: Fajr (4 rakats total), Dhuhr (12), Asr (8), Maghrib (7), and Isha (15 including Witr). Complete bilingual English/Urdu content is provided with proper RTL rendering for Urdu text.

3.8 Mosque Map Screen (mosque_map_screen.dart)

The MosqueMapScreen (approximately 750 lines) provides a feature-rich nearby mosque finder with interactive mapping.

Map Engine and Data Sources:

Built on the flutter_map package (OpenStreetMap-based) with MapController for programmatic map control. The screen supports dark mode map tiles (CartoDB dark tiles) and standard OSM tiles for light mode. Mosque data is fetched using a dual API strategy: Nominatim is tried first, then Overpass API as a fallback, ensuring redundancy.

Dynamic Radius Search:

The mosque discovery algorithm progressively expands the search radius (3km, 10km, 25km, 50km) until at least 3 mosques are found. The Overpass query searches for three tag combinations: amenity=place_of_worship + religion=muslim, amenity=mosque, and building=mosque. A Set-based deduplication system using "name_lat" key patterns filters out duplicate entries.

Routing and Navigation:

The OSRM routing API provides walking and driving routes rendered as polylines on the map. A travel mode toggle allows switching between foot and car profiles. Routes are lazily loaded — only the nearest 3 mosques get routes initially, with others loading on selection. Custom location support allows long-pressing the map to set a different origin point, with a "Reset to GPS" button to revert. External navigation launches Google Maps via url_launcher for turn-by-turn directions.

3.9 Notification Service (notification_service.dart)

The NotificationService (approximately 415 lines) is implemented as a static utility class with dual notification channels.

Dual Channel Architecture:

Two notification channels are configured: "azan_channel" uses the system default sound for prayer time banners (non-intrusive), while "reminder_azan_channel" uses custom Azan audio (azan.mp3) that auto-plays when reminders fire. The channel refresh strategy deletes old channels before recreating them to force Android to pick up sound changes.

Background Azan Playback:

A critical feature is the azanAlarmCallback() function — a top-level function annotated with @pragma("vm:entry-point") that is fired by AndroidAlarmManager even when the app is completely closed. This callback uses AudioPlayer from the audioplayers package to play the Azan audio for 30 seconds. The alarm is scheduled using AndroidAlarmManager.oneShotAt() with exact: true, wakeup: true, and rescheduleOnReboot: true flags.

Cross-Platform Support:

iOS notification support is provided via DarwinNotificationDetails with the Azan sound file. Timezone handling uses flutter_timezone to detect the local timezone with a UTC fallback. Permission handling covers both Android notification permissions and exact alarm permissions. A daily Hadith notification can be scheduled at a configurable hour with language-aware content.

3.10 Splash Screen (splash_screen.dart)

The SplashScreen (approximately 500 lines) provides a cinematic, multi-layered animated introduction with three custom CustomPainter classes:

Animation Architecture:

Two AnimationControllers drive the animations: a main controller (2800ms, one-shot) drives four staggered Interval animations — bead convergence (0-60%), emblem reveal (35-78%), light wave (48-88%), and background shift (30-85%); and a rotation controller (14s loop, infinite) provides continuous bead rotation with a counter-rotating emblem.

Visual Elements:

PremiumSplashBackdropPainter renders an architectural mosque silhouette with a starfield background.

CinematicSplashPainter animates 33 tasbeeh beads converging from scattered positions (calculated via $\sin(i*7.3)$ and $\cos(i*3.7)$ offsets) to a perfect circle using linear interpolation. EmblemFramePainter draws a rotating Rub el Hizb 8-point star frame (two concentric squares rotated by $\pi/4$ with $\sqrt{2}$ distance scaling). RepaintBoundary widgets isolate painting layers to minimize repaints for performance. The route transition uses a PageRouteBuilder with FadeTransition (1100ms) to cross-fade into MainWrapper.

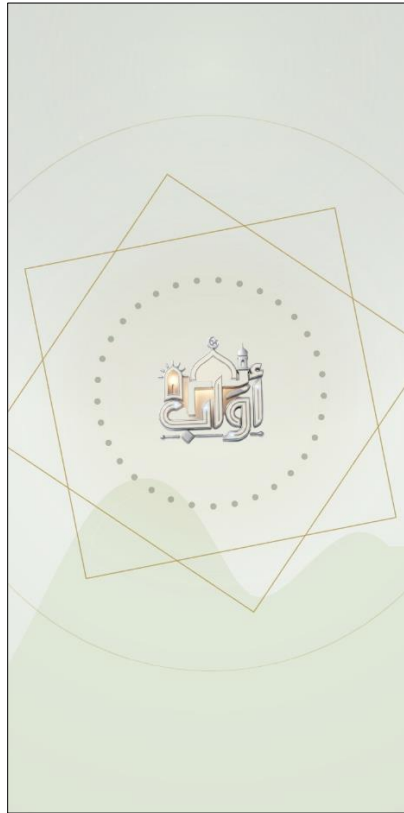
3.11 Settings Screen (settings_screen.dart)

The SettingsScreen (approximately 244 lines) provides four primary personalization options: (1) a Dark Mode toggle that switches the theme via ThemeNotifier.toggleTheme() from the Provider; (2) a Language selector dropdown (English/Urdu) that calls LanguageProvider.setLanguage(), instantly updating all UI labels throughout the app; (3) an Azan Alerts toggle — when enabled, the screen fetches prayer times (from cache or fresh via GPS), cancels any existing notifications, and schedules all five prayers with NotificationService.scheduleNotification() including the Azan sound; when disabled, all prayer notifications are cancelled; and (4) an About dialog using Flutter's built-in showAboutDialog() with the app description, version, and developer credits. Error handling shows a SnackBar on failure and resets the switch state.

4. Output Screenshots

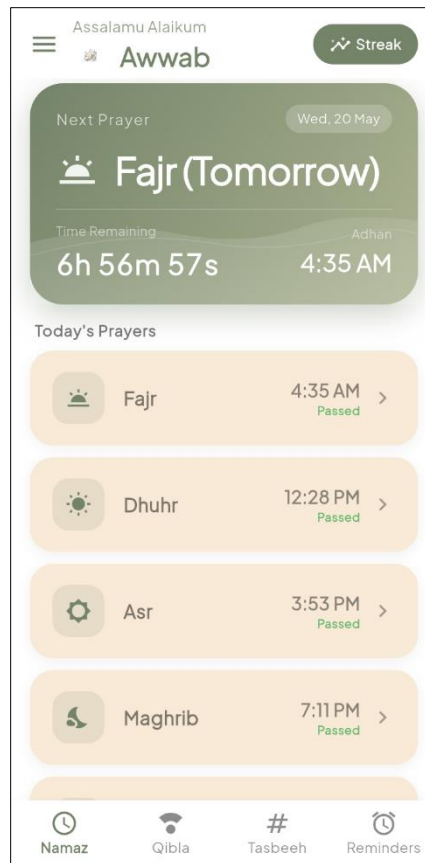
Screenshot 1 — Splash Screen

The cinematic splash screen displaying animated tasbeeh beads converging into a circle, the Rub el Hizb star frame, and the Awwab logo with a dark gradient background.



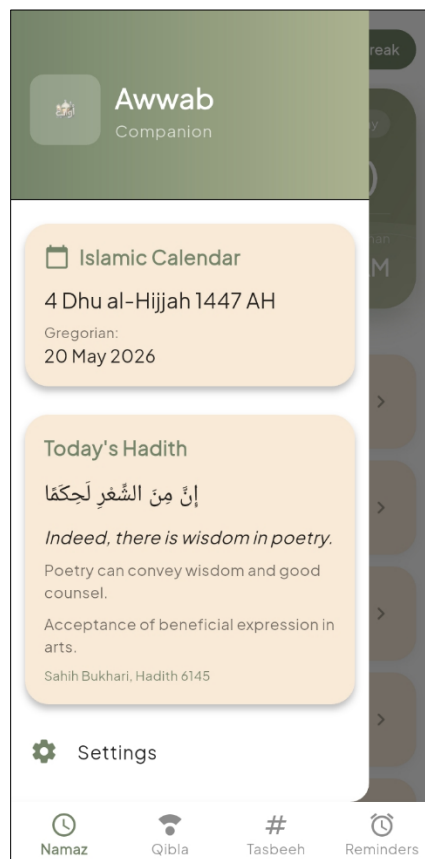
Screenshot 2 — Home Screen (Prayer Times)

The main prayer times screen showing all five daily prayers with times in 12-hour format, the next prayer highlighted with a border/shadow, and the countdown timer hero card with mosque silhouette backdrop.



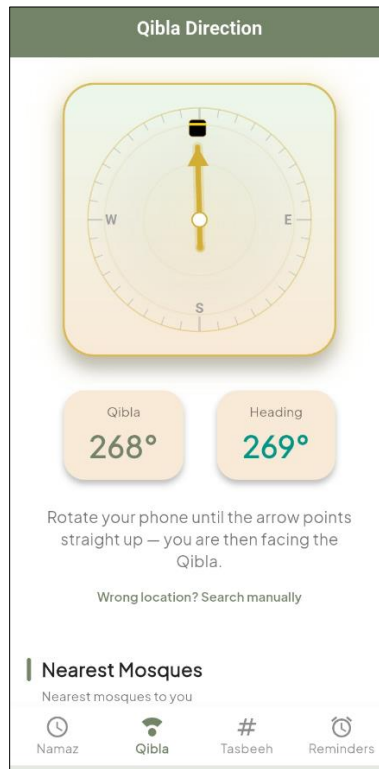
Screenshot 3 — Side Drawer

The slide-out drawer showing the app branding header, Hijri/Gregorian calendar dates, and the daily Hadith card with Arabic text, translation, and Tafseer.



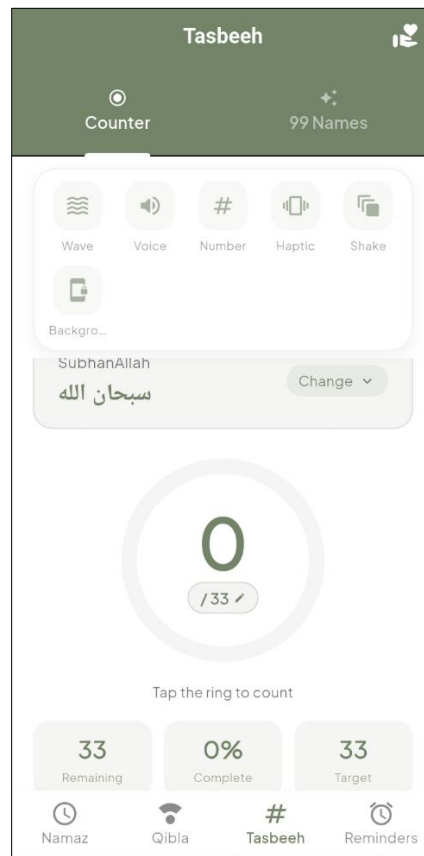
Screenshot 4 — Qibla Compass

The Qibla direction screen with the custom-painted compass rose, cardinal directions, degree markings, Qibla needle pointing toward Makkah, and the bearing angle.



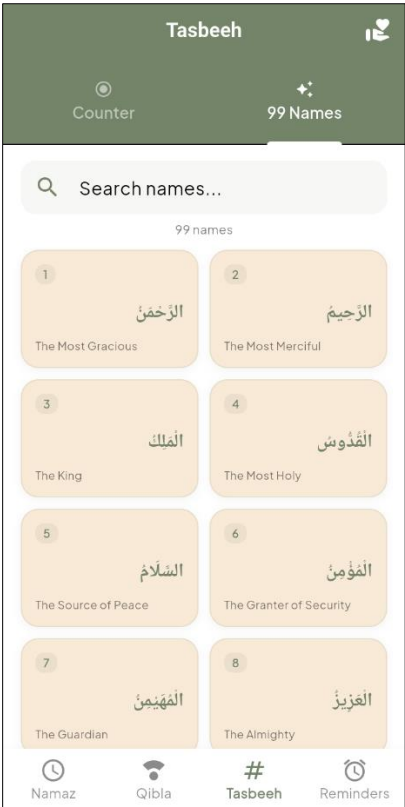
Screenshot 5 — Tasbeeh Counter

The digital dhikr counter showing the circular progress ring (CustomPainter), count display, and the preset selection bottom sheet.



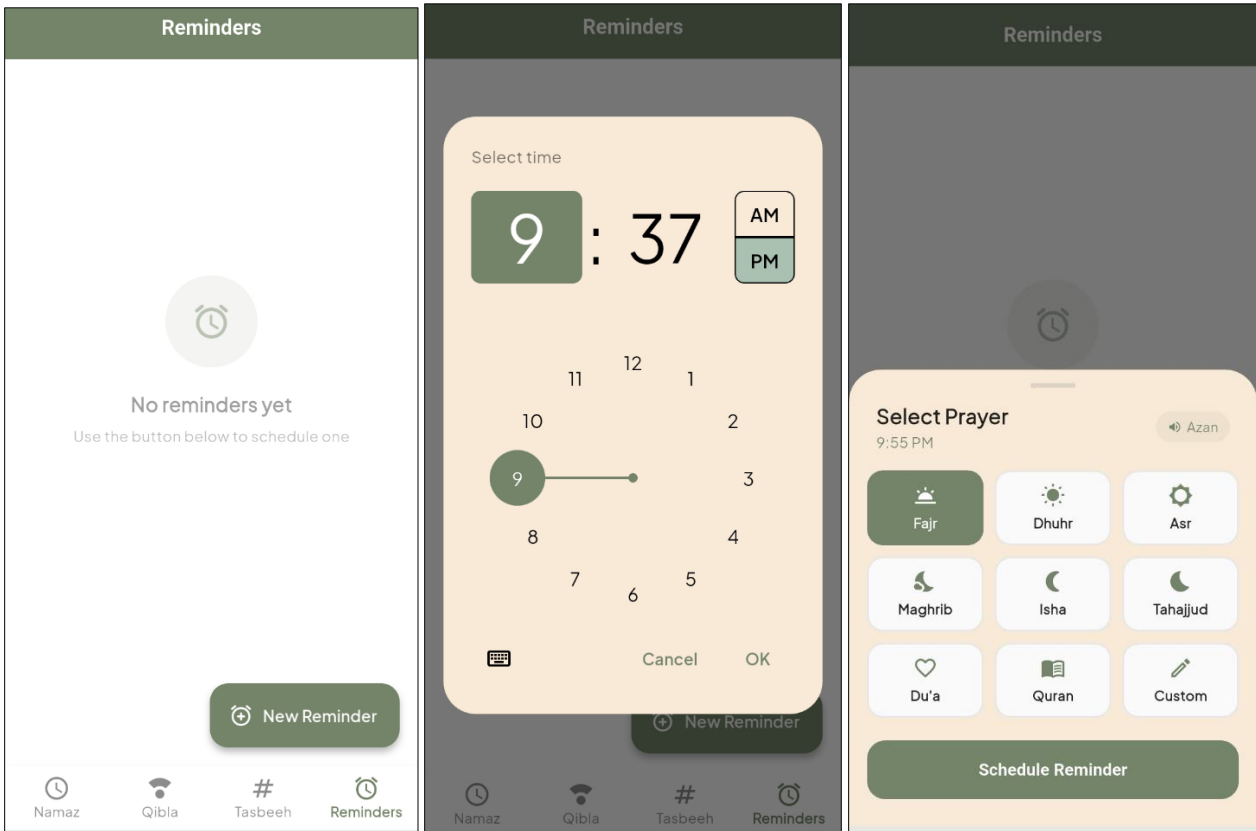
Screenshot 6 — 99 Names of Allah

The Asma ul Husna tab displaying the scrollable list with Arabic text, English and Urdu meanings, and the live search functionality.



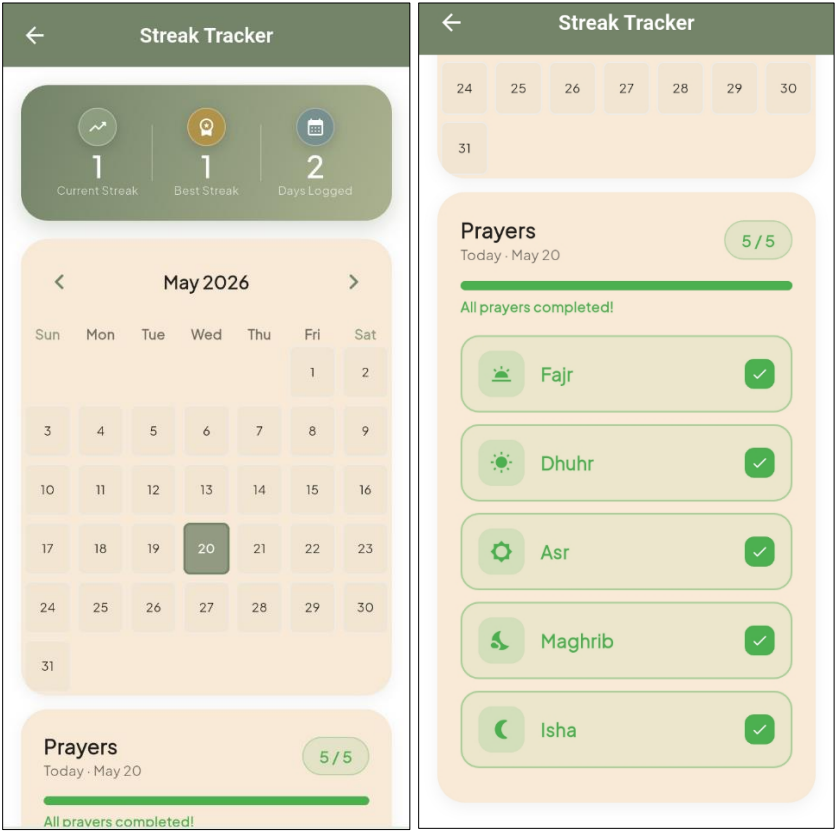
Screenshot 7 — Reminders

The reminders screen showing user-created reminder entries with prayer labels, scheduled times, and swipe-to-delete functionality.



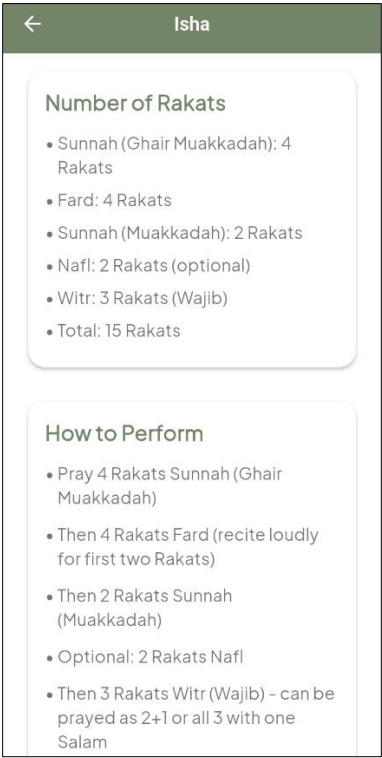
Screenshot 8 — Streak Tracker

The streak tracker showing the monthly calendar heatmap with colored completion markers, streak counters, and the daily prayer checkboxes.



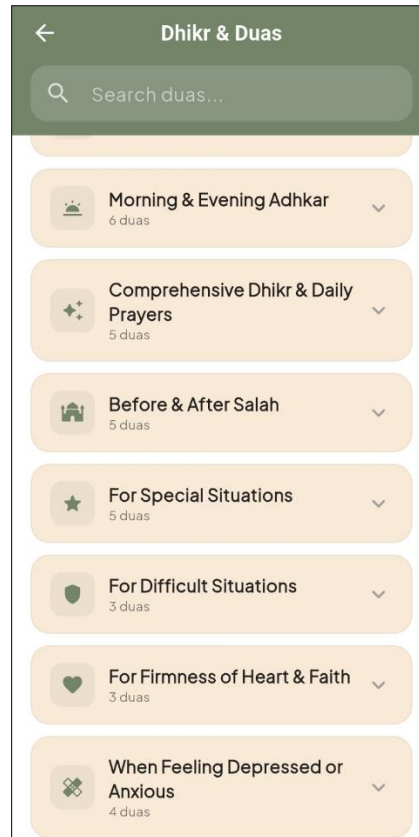
Screenshot 9 — Prayer Guidance

The step-by-step prayer guidance showing the rakat breakdown table and numbered instruction steps for a selected prayer.



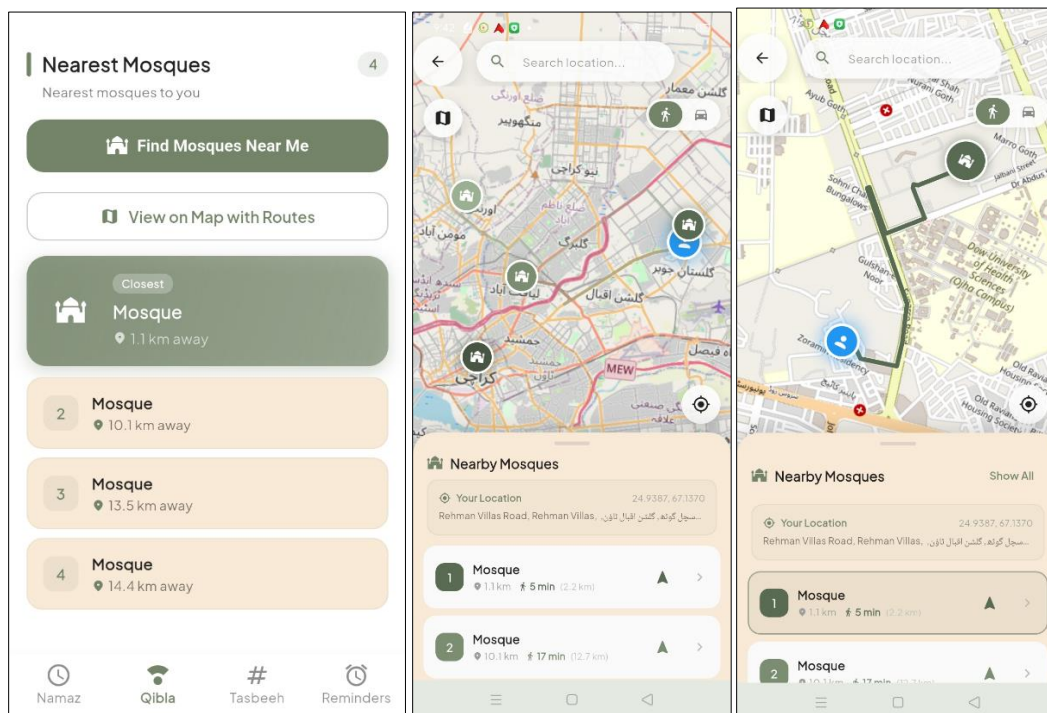
Screenshot 10 — Dua & Dhikr Guidance

The guidance hub showing expandable sections of Islamic duas with Arabic text, translations, and Hadith references.



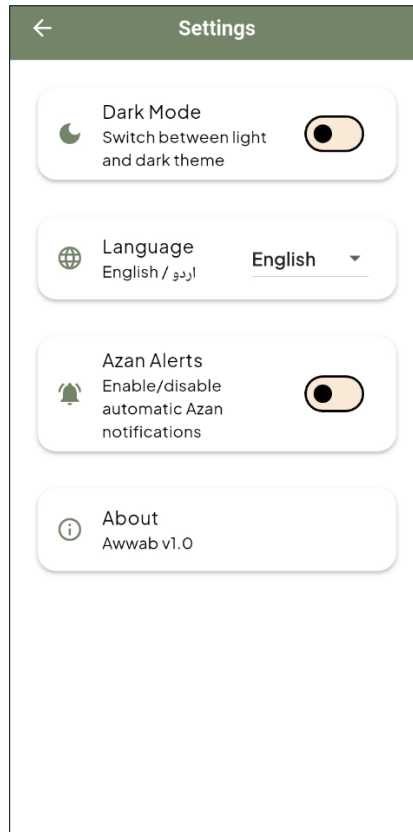
Screenshot 11 — Mosque Map

The nearby mosque finder with OpenStreetMap tiles, mosque markers, route polylines, and the bottom sheet listing mosques with distances.



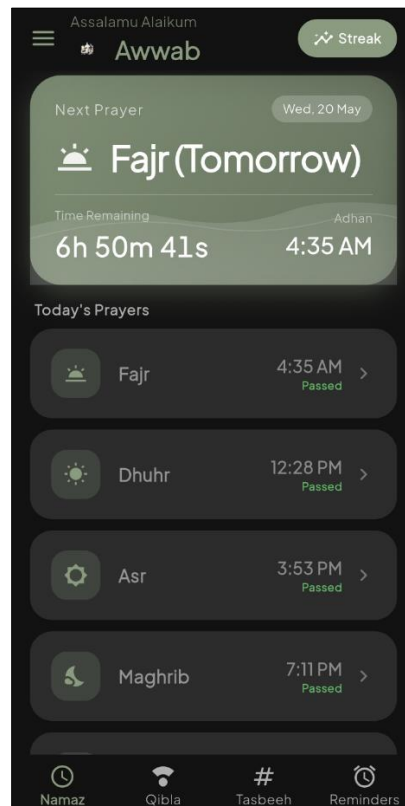
Screenshot 12 — Settings

The settings screen showing the dark mode toggle, language dropdown, and Azan alerts toggle.



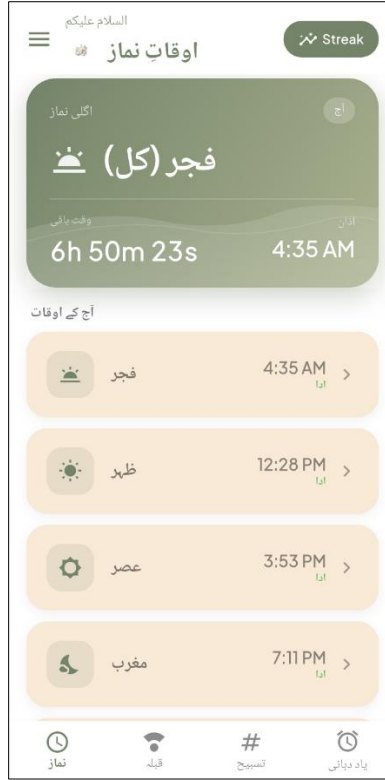
Screenshot 13 — Dark Mode

The Home Screen in dark mode demonstrating the deep dark background with olive accents and proper text contrast.



Screenshot 14 — Urdu Language

The Home Screen in Urdu language mode demonstrating RTL layout and Arabic script rendering for prayer names and UI labels.



5. Challenges and Solutions

During the development of Awwab, the team encountered several significant technical challenges. This section documents each challenge and the solution that was implemented.

5.1 Accurate Prayer Time Calculation Across Time Zones

Challenge:

Calculating accurate prayer times requires precise handling of the user's geographic location and time zone. Different cities — even within the same country — can have significantly different prayer times, and daylight saving time changes add further complexity. Additionally, the app needed to display times immediately on launch before the network request could complete.

Solution:

We integrated the Aladhan API, which uses astronomically-precise calculations based on GPS coordinates with the ISNA (Islamic Society of North America) calculation method. To handle time zones correctly, we used the `timezone` and `flutter_timezone` packages to initialize timezone-aware `TZDateTime` objects. The `LocationService` wraps the `Geolocator` plugin with proper permission handling and fallback behavior. A two-phase loading strategy was implemented: cached prayer times are loaded instantly from `SharedPreferences` on launch, while fresh times are fetched from the API after a 500ms delay and the cache is updated.

5.2 Reliable Background Notification Scheduling

Challenge:

Android's battery optimization (Doze mode) aggressively kills background processes and delays `AlarmManager` alarms. This made it difficult to ensure Azan notifications fire at the exact scheduled prayer time, especially when the app is closed or the device is idle. Additionally, the Azan audio needed to play even without user interaction.

Solution:

We implemented a dual notification channel system with background Azan playback. The `android_alarm_manager_plus` plugin leverages Android's `setExactAndAllowWhileIdle()` API for precise alarms during Doze mode. A top-level `@pragma("vm:entry-point")` callback function (`azanAlarmCallback`) is invoked by the `AlarmManager` even when the app is completely closed — this callback plays the Azan audio for 30 seconds via `AudioPlayer`. The `RECEIVE_BOOT_COMPLETED` permission ensures alarms are rescheduled after device reboot. The channel refresh strategy deletes and recreates channels to force Android to pick up sound configuration changes.

5.3 Real-Time Qibla Compass Accuracy

Challenge:

The device's magnetometer sensor can be inaccurate due to magnetic interference, uncalibrated sensors, or proximity to electronic devices. This leads to the compass needle pointing in the wrong direction, which is particularly problematic for a prayer direction finder that users rely on for correct prayer orientation.

Solution:

We implemented a calibration detection system that monitors compass accuracy from the `flutter_compass` plugin. When low accuracy is detected, a calibration dialog instructs the user to perform a "figure-8" motion. The Qibla bearing is calculated using the Haversine-based bearing formula relative to the Kaabah coordinates (21.4225 N, 39.8262 E). The compass animation uses `AnimatedRotation` with interpolation to smooth out noisy sensor data. The custom `QiblaCompassPainter` renders a detailed compass rose with degree markings and a Kaaba visualization at the Qibla bearing.

5.4 Bilingual Support with RTL Layout

Challenge:

Supporting both English (LTR) and Urdu (RTL) throughout the entire app required careful handling of text alignment, layout direction, and content switching. Urdu uses Arabic script with right-to-left rendering and different typographic requirements. The switch needed to be instant and affect every screen simultaneously.

Solution:

We implemented a global `LanguageProvider` using the `Provider` package with a language code-based approach. Every user-facing string uses a ternary expression checking the `isUrdu` flag. The `Provider's ChangeNotifier` pattern ensures instant, app-wide updates. Typography uses Google Fonts with `Plus Jakarta Sans` for English and `Noto Sans Arabic` for Arabic/Urdu text, configured through the centralized `AppTheme`. All prayer guidance content (10 `dua/dhikr` sections, 5 prayer methods, `rakat` breakdowns), 99 Names of Allah, navigation labels, reminder labels, and UI strings have complete bilingual versions.

5.5 Offline Functionality and Data Persistence

Challenge:

The app needed to function effectively without an internet connection while persisting user data like prayer completion streaks, notification preferences, and theme/language settings. Additionally, frequent reads of streak data during calendar scrolling needed to be performant.

Solution:

We implemented a two-tier caching strategy: an in-memory cache in `StreakService` keeps the full prayer records map in RAM after the first load — all reads hit the cache while only writes go to `SharedPreferences` disk storage. "Fire-and-forget" writes (calling `setString()` without `await`) prevent blocking the UI thread. The Hadith dataset is bundled as a local JSON asset. Prayer times are cached to `SharedPreferences` for instant display on launch. Legacy format migration is handled transparently — if old List-format data is detected, it is cleared and reset.

5.6 Cross-Platform Compatibility

Challenge:

Flutter supports six platforms, but platform-specific features like notification scheduling, compass sensors, GPS, alarm managers, and map rendering behave differently on each platform. Android-specific alarm APIs, iOS background modes, and web PWA configurations all needed separate handling.

Solution:

We used well-maintained Flutter plugins that abstract platform differences. Platform-specific configurations include: `AndroidManifest.xml` (permissions, hardware features, boot receiver), `Info.plist` (background modes: fetch and remote-notification, location usage descriptions), and `manifest.json` (PWA configuration with maskable icons). `DarwinNotificationDetails` provides iOS-specific notification configuration. Map rendering uses `flutter_map` with different tile providers for light mode (OSM) and dark mode (CartoDB dark tiles). The `android_alarm_manager_plus` plugin handles Android's exact alarm scheduling while iOS uses its native notification capabilities.

5.7 Performant UI Animations

Challenge:

The app features numerous simultaneous animations (splash screen with 33 animated beads, compass rotation, counter pulse, streak checkmarks, tab transitions) that need to run smoothly at 60 FPS without jank or excessive battery drain.

Solution:

We leveraged Flutter's GPU-accelerated animation framework. The splash screen uses RepaintBoundary widgets to isolate painting layers, minimizing repaints. Staggered Interval animations within a single AnimationController coordinate multiple visual elements without multiple controllers. CustomPainter classes render complex visuals (compass rose, progress ring, beads, star frame) directly on the Canvas for maximum efficiency. Implicit animations (AnimatedContainer, AnimatedOpacity) handle simple transitions declaratively. TweenAnimationBuilder provides GPU-thread animation without manual controller management.

5.8 Comprehensive Mosque Discovery

Challenge:

Finding nearby mosques required access to a worldwide mosque database. Commercial map APIs (Google Maps) require API keys and incur costs. Additionally, mosque data completeness varies by region, and the search needed to be robust enough to find results even in areas with sparse data.

Solution:

We implemented a multi-strategy approach: a dual API fallback system (Nominatim first, then Overpass) ensures redundancy. A dynamic radius search progressively expands from 3km to 50km until at least 3 mosques are found. Three OpenStreetMap tag combinations are searched simultaneously. OSRM provides walking/driving route calculations rendered as map polylines. Lazy route loading (only nearest 3 initially) optimizes network usage. Set-based deduplication prevents duplicate entries. Custom location support allows users to search from any map point, not just their GPS position. All services are completely free with no API keys or usage limits.

5.9 Technology Stack Summary

The following table summarizes all major technologies, packages, and dependencies used in the Awwab application:

Category	Technology/Package	Version	Purpose
Framework	Flutter	>=3.38.4	Cross-platform UI framework
Language	Dart	>=3.10.8	Programming language
State Mgmt	provider	^6.0.5	App-wide state management
API	http	^1.1.0	HTTP requests to Aladhan API
Location	geolocator	^13.0.2	GPS position retrieval
Compass	flutter_compass	^0.8.0	Real-time compass heading
Sensors	sensors_plus	^6.0.1	Device sensor access
Notifications	flutter_local_notifications	^17.2.1	Local push notifications
Alarms	android_alarm_manager_plus	^4.0.3	Exact-time background alarms
Audio	audioplayers	^6.6.0	Azan sound playback
Storage	shared_preferences	^2.2.2	Key-value local storage
Calendar	table_calendar	^3.0.9	Streak tracker calendar widget
Hijri Date	hijri_date	^1.0.1	Islamic calendar conversion
Timezone	timezone	^0.9.4	Timezone-aware scheduling
TZ Detect	flutter_timezone	^3.0.1	Local timezone detection
Date Format	intl	^0.18.1	Date/time formatting
Permissions	permission_handler	^11.3.1	Runtime permission requests
Maps	flutter_map	^8.3.0	OpenStreetMap rendering
Coordinates	latlong2	^0.9.1	Geographic coordinate handling
URL Launch	url_launcher	^6.3.2	Open external map apps
Fonts	google_fonts	^6.1.0	Plus Jakarta Sans + Noto Sans Arabic
Icons	cupertino_icons	^1.0.8	iOS-style icon set
SVG	flutter_svg	^2.0.9	SVG rendering support
TTS	flutter_tts	^4.2.5	Text-to-speech capability
Proximity	proximity_sensor	^1.3.10	Proximity sensor access

Vibration	vibration	^3.1.8	Haptic feedback
Wake Lock	wakelock_plus	^1.6.0	Keep screen awake
Volume	flutter_volume_controller	^1.3.4	Audio volume control
Splash	flutter_native_splash	^2.4.0	Native splash screen generation
App Icons	flutter_launcher_icons	^0.14.3	App icon generation
Linting	flutter_lints	^6.0.0	Code quality rules